

AIP 로봇 응용 SW 면접 복습용 학습 자료

클로봇 로봇 응용 SW 개발자 공고 기준
ROS2 - 웹관제 - 시뮬레이션 - 실차 확장 흐름 정리

기준	내용
목적	면접 전에 프로젝트 구조와 핵심 개념을 빠르게 복습
범위	현재 저장소와 사용자가 제공한 공고 기준, 외부 자료 미사용
원칙	확인된 기능과 확인 필요한 기능을 분리해서 설명
데모 주소	AIP 웹관제 http://localhost:18080 , Foxglove ws://localhost:18765

읽는 순서: 개념 -> 프로젝트 구조 -> 데이터 흐름 -> 시뮬레이션 -> 실차 확장 -> 면접 답변

생성일: 2026-07-06

1. 공고 요구사항과 프로젝트 연결

이 공고는 서비스 로봇 SW, 센서 연동, 로봇 관제 대시보드, UI, 테스트와 배포를 함께 본다. AIP 프로젝트는 제품 수준 완성보다 로봇 응용 SW의 연결 구조를 설명하기에 좋은 소재다.

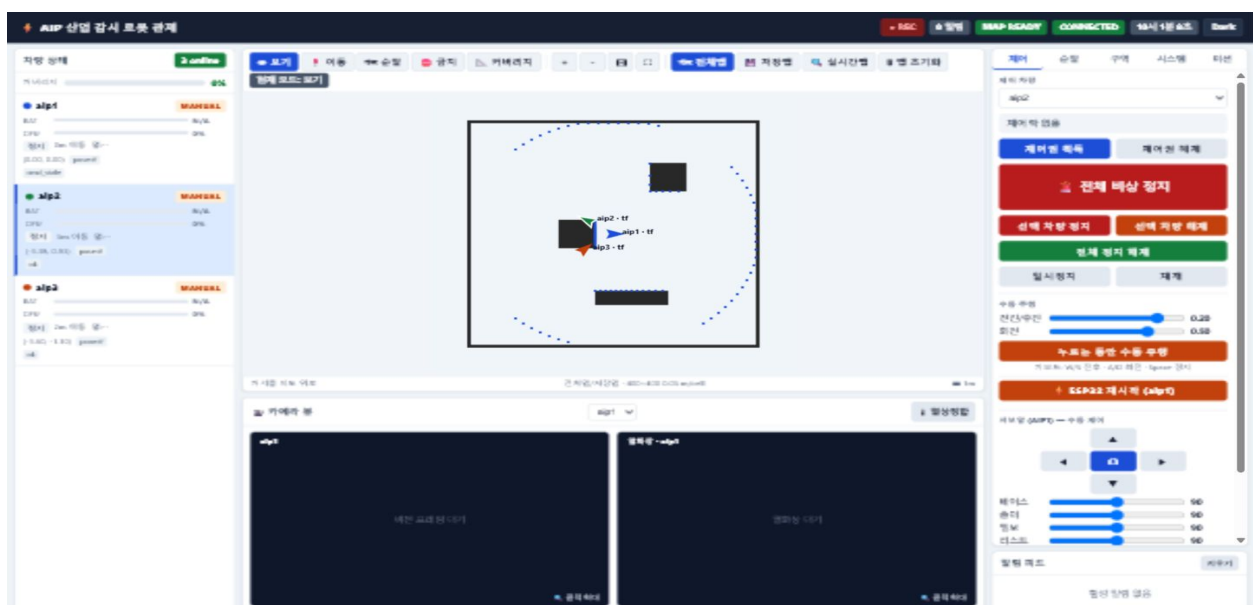
공고 키워드	AIP에서 설명할 수 있는 근거	면접 표현
ROS 경험	ROS2 Humble, rclpy/rclcpp, custom msg/srv, Nav2 ActionClient	Topic 계약과 노드 역할 중심으로 설명
웹-로봇 연동	FastAPI dashboard_server가 ROS2와 WebSocket 사이를 중계	브라우저가 직접 ROS에 붙지 않고 backend가 bridge 역할
센서 연동	LiDAR scan, odom, RGB/thermal image, alert topic 경로	코드 경로는 확인됨. 현장 성능은 확인 필요로 분리
제어 기능	twist_mux, cmd_vel, override_cmd_vel, estop, watchdog	수동 명령과 autonomy 명령의 우선순위 체인을 설명
검증/배포	Docker sim, 포트 분리, PDF/README/docs 정리	실행 가능한 시물 데모와 한계 정리

핵심 메시지: 완성된 상용 로봇이 아니라, ROS2 기반 다중 로봇 관제 구조를 시뮬레이션으로 검증하고 실차 확장 인터페이스를 정리한 프로젝트라고 말한다.

2. 전체 시스템 한 장 요약

Browser UI	->	dashboard_server	->	ROS2 fleet topics	->	vehicle nodes
------------	----	------------------	----	-------------------	----	---------------

영역	주요 구성	역할
중앙 PC	supervisor, watchdog, dashboard, coordinator, perception	상태 집계, 안전 감시, 웹관제, follower 명령 생성
공통 Topic	/fleet/status, /fleet/override, /fleet/alerts, /fleet/peer_poses	차량별 데이터를 fleet 단위로 묶는 계약
차량	aip1, aip2, aip3 namespace	heartbeat, odom, scan, cmd_vel 계약을 차량별로 분리
시뮬레이션	aip_fleet_sim, demo_patrol_node, world.yaml	실차 없이 웹관제와 ROS2 흐름을 보여주는 데모
실차 확장	fleet_main, turtlebot3, custom_vehicle, ESP32 firmware	하드웨어별 driver만 바꾸고 상위 계약은 유지



현재 확인 범위: 웹관제, 3대 시뮬 상태 표시, map ready, 데모 주행, 포트 분리. 실차 3대 완전 자율주행은 확인 필요.

3. ROS2 핵심 개념

AIP를 설명할 때는 ROS2를 추상적으로 말하기보다, 어떤 노드가 어떤 Topic, Service, Action을 쓰는지 연결해서 말하는 것이 좋다.

개념	의미	AIP 예시
Node	기능 단위 실행 프로세스	supervisor_node, dashboard_server, sim_vehicle_node
Topic	연속 데이터 publish/subscribe	/aip1/heartbeat, /fleet/status, /aip1/cmd_vel
Message	Topic 데이터 형식	FleetHeartbeat, FleetStatus, OverrideCommand, Twist
Service	요청-응답형 호출	/save_map_now, costmap clear service
Action	오래 걸리는 목표 수행	/aipN/navigate_to_pose Nav2 ActionClient
TF	좌표계 변환	odom -> base_link, map 기준 pose 표시
QoS	통신 신뢰성과 유지 정책	map/heartbeat/scan 특성에 따라 조정 필요

면접 포인트: Nav2를 직접 구현했다고 말하기보다, NavigateToPose ActionClient로 goal을 전달하고 관제 UI에서 그 흐름을 연결했다고 설명한다.

4. AIP 패키지 구조와 역할

패키지/영역	역할	면접에서 강조할 점
aip_fleet_msgs	공통 msg/srv 정의	차량이 달라도 상태/제어 계약을 통일
aip_fleet_supervisor	heartbeat 집계와 watchdog	offline 차량을 감지해 안전 명령으로 연결
aip_fleet_dashboard	FastAPI + WebSocket 웹관제	ROS2와 브라우저 사이의 bridge
aip_fleet_sim	2D kinematic sim, map, lidar, vehicle	실차 없이 관제 흐름 검증
aip_fleet_real	serial bridge, heartbeat, TurtleBot3/custom launch	실차별 driver 연결 지점
aip_fleet_perception	RGB/thermal/alert 처리	센서 데이터를 관제 화면까지 전달
aip_fleet_coordinator	follower coord_cmd_vel 생성	중앙 의존 구조와 향후 개선을 함께 설명

프로젝트를 설명할 때는 패키지 이름을 외우는 것보다, '공통 계약 - 중앙 집계 - 대시보드 - 차량 driver' 계층으로 묶어 설명하면 안정적이다.

5. 상태 데이터 흐름

/<vid>/heartbeat	->	supervisor_node	->	/fleet/status	->	dashboard_server	->	Browser
------------------	----	-----------------	----	---------------	----	------------------	----	---------

상태 흐름은 차량이 살아 있는지, 어떤 모드인지, E-Stop인지, battery/status가 어떤지 관제 화면에 보여주는 기본 경로다.

단계	데이터	확인 방법
차량 또는 sim	FleetHeartbeat 발행	ros2 topic echo /aip1/heartbeat --once
supervisor	여러 heartbeat를 FleetStatus로 집계	ros2 topic echo /fleet/status --once
dashboard_server	FleetStatus를 WebSocket JSON으로 변환	브라우저에서 3 online, AUTO/ESTOP 표시
브라우저	카드, 지도 marker, 알람 UI 갱신	http://localhost:18080

주의: 실차는 heartbeat 메시지 필드 정합이 중요하다. ESP32 scout firmware 쪽은 현재 메시지 필드 정합 수정이 필요할 수 있다.

6. 제어 데이터 흐름

Browser command	->	WebSocket /ws	->	dashboard_server	->	/fleet/override	->	supervisor
supervisor	->	/<vid>/override_cmd_vel	->	twist_mux	->	/<vid>/cmd_vel	->	driver

입력	Topic/경로	역할
수동 override	/<vid>/override_cmd_vel	운영자가 임시로 차량 속도 명령
autonomy	/<vid>/autonomy_cmd_vel	Nav2 또는 데모 주행 명령
coordination	/<vid>/coord_cmd_vel	follower 차량 추종 명령
E-Stop	/<vid>/estop, /fleet/override	정지와 안전 상태 전달
최종 출력	/<vid>/cmd_vel	driver 또는 sim_vehicle_node가 실제로 소비

핵심 표현: 여러 제어 입력을 바로 모터에 보내지 않고, twist_mux 우선순위 체인을 거쳐 최종 cmd_vel로 합친다.

7. 웹관제 구조

메인 웹관제는 React/Vue가 아니라 FastAPI backend와 정적 HTML/JavaScript 기반이다. Foxglove React/TypeScript 패널은 별도 보조 도구로 구분해서 말한다.

구성	확인된 역할	면접 설명
FastAPI	GET /, /static, /ws 제공	브라우저 UI와 ROS2 사이의 서버
WebSocket	상태 push와 제어 command 처리	실시간 관제에 필요한 양방향 통신
Canvas map	OccupancyGrid, pose, scan 표시	지도 위 차량 상태를 시각화
Control UI	E-Stop, release, override, waypoint, keepout	관제 명령이 ROS2 Topic/Action으로 변환
Vision panel	RGB/thermal/perception frame 표시	센서 화면을 관제 화면에 통합

rosbridge/roslibjs 사용은 현재 코드에서 확인되지 않았다. 따라서 'FastAPI 기반 WebSocket bridge'라고 표현한다.

8. 시뮬레이션 구조

현재 로봇이 없는 상황에서도 웹관제를 시연할 수 있도록 Docker sim이 정리되어 있다. 내부 서버 포트는 8080이지만 host 접속은 수업용 stack과 충돌하지 않도록 18080으로 바꿨다.

구성	역할	현재 상태
docker/sim	시뮬 전용 컨테이너와 포트 매핑	http://localhost:18080, ws://localhost:18765
sim_world_node	map, map_static, map_ready 발행	창고형 obstacle map 표시
sim_vehicle_node	cmd_vel 적분, odom, heartbeat, TF 발행	3대 차량 상태 표시
demo_patrol_node	aip1에 autonomy_cmd_vel 발행	시뮬 전용 자동 데모 주행
coordinator	aip2/aip3 follower 명령	데모에서 추종 흐름 설명 가능

중요: demo_patrol_node는 aip_fleet_sim launch에만 포함된다. 실제 차량 bringup에는 포함되지 않으므로 시뮬/실차 구분이 되어 있다.

9. 실제 차량 확장 전략

차량	예상 구성	현재 설명 가능 범위
aip1	Pi 기반 메인 AGV, LiDAR, serial_bridge, SLAM/Nav2	기존 코드와 문서 근거 있음. 현재 환경에서는 실차 직접 검증 범위 분리
aip2	TurtleBot3 bringup	turtlebot3.launch.py 구조 확인. 실제 장비 상태는 확인 필요
aip3	커스텀 ESP32/micro-ROS 또는 STS3215 차량	custom_vehicle placeholder와 scout firmware skeleton 확인
공통 계약	/<ns>/heartbeat, /<ns>/cmd_vel, /<ns>/odom, /<ns>/scan, /<ns>/estop	차량이 달라도 dashboard/supervisor는 같은 계약을 본다

면접에서는 '이기종 로봇을 같은 관제 인터페이스로 묶기 위해 namespace와 Topic 계약을 맞췄다'고 설명한다.

실차 3대 동시 완전 자율주행은 아직 확인 필요하다. 시뮬레이션, 부분 실차, launch/config 구조를 구분해서 말한다.

10. ESP32와 Pi 없는 상황

ESP32(모터), ESP32-S3(LiDAR), ESP32_CAM(카메라)만으로도 관제 연동 프로토타입은 가능하다. 다만 Pi 없이 SLAM/Nav2 전체를 차량 안에서 돌리는 것은 어렵다.

목표	가능성	조건
웹관제 online 표시	가능	heartbeat를 micro-ROS 또는 bridge로 발행
수동 주행	가능	/aip3/cmd_vel 수신 후 motor PWM 매핑 구현
E-Stop	가능	/aip3/estop 수신 후 모터 hard stop 구현
LiDAR 표시	조건부 가능	ESP32-S3가 LaserScan 또는 raw scan을 안정 발행해야 함
카메라 표시	가능	ESP32_CAM HTTP MJPEG/JPEG를 dashboard stream 또는 bridge로 연결
Nav2 자율주행	제한적	중앙 PC가 compute brain 역할. latency와 odom 품질 검증 필요

추천 단계: heartbeat -> cmd_vel/estop -> odom -> scan -> camera -> 중앙 PC Nav2 시도 순서로 검증한다.

11. 센서와 비전 연동

센서	ROS2 또는 웹 경로	설명 포인트
LiDAR	/<vid>/scan	지도, 장애물, Nav2 입력. 실차 모델별 driver 설정 필요
Odometry	/<vid>/odom, TF	차량 위치 추정과 지도 marker 표시의 기본
RGB Camera	CompressedImage 또는 HTTP stream	Vision Pi bridge와 dashboard 표시 경로 확인
Thermal	thermal_raw, thermal_viz, /fleet/alerts	온도 기반 alert와 visualization
Perception	/fleet/perception_viz/<vid>, /fleet/alerts	탐지 결과를 관제 UI에 보여주는 흐름

YOLOv8 호출 코드와 alert fusion 구조는 확인되지만, 현장 정확도나 fire/smoke 전용 모델 검증은 확인 필요로 말한다.

ESP32_CAM은 우선 HTTP stream으로 붙이고, ROS2 image topic이 필요하면 중앙 PC에서 HTTP-to-ROS bridge를 두는 접근이 현실적이다.

12. 안전 구조

로봇 관제에서는 움직이는 기능보다 멈추는 기능을 먼저 설명할 수 있어야 한다. AIP는 여러 정지 경로를 분리해 설명할 수 있다.

안전 요소	현재 코드 근거	면접 표현
Web E-Stop	index.html, dashboard_server.py	운영자 버튼이 ROS2 estop/override로 변환
Watchdog	watchdog_node.py	heartbeat offline이 지속되면 ESTOP 발행
Supervisor route	supervisor_node.py	fleet override를 차량별 estop/zero Twist로 변환
Command stale	sim_vehicle_node.py	명령이 오래 없으면 속도 0 처리
Motor watchdog	firmware/main_agv/motor_control.cpp	ESP32 구동계도 timeout 시 PWM 0
Control lock	dashboard/supervisor	운영자 제어권 충돌을 줄이기 위한 구조

주의: 실제 물리 E-Stop 입력과 모든 차량의 estop_lock 동작은 차량별 검증 필요로 분리한다.

13. 실행과 검증 체크리스트

데모 전에 아래 순서로 확인하면 면접이나 발표 중 흔들릴 가능성이 줄어든다.

```
docker compose -f docker/sim/docker-compose.yml up -d --build
```

확인 항목	기준	명령 또는 화면
수업 포트 보호	Spark 8080 유지, AIP 18080 사용	브라우저 localhost:8080과 localhost:18080 구분
웹관제 접속	CONNECTED	http://localhost:18080
맵 준비	MAP READY	/fleet/map_ready 또는 화면 badge
차량 상태	3 online	/fleet/status
주행 표시	aip1 speed와 odom 증가	/aip1/cmd_vel, /aip1/odom
Foxglove	18765 open	ws://localhost:18765
문서 표현	확인됨/확인 필요 분리	README, PDF, 포트폴리오 설명

14. 면접 답변 프레임

질문	답변 프레임
프로젝트를 한 문장으로 설명하면?	ROS2 기반 다중 로봇 관제 구조를 시뮬레이션으로 검증하고, 실차 확장을 위한 Topic 계약과 웹관제 흐름을 정리한 프로젝트입니다.
본인 역할은?	웹관제, ROS2 통신 계약, 서버차량 제어 흐름, 시뮬레이션 데모, 문서화를 중심으로 설명합니다.
웹과 로봇은 어떻게 연결되나요?	브라우저가 직접 ROS2에 붙지 않고 FastAPI dashboard_server가 ROS2 Topic/Action을 WebSocket 메시지로 변환합니다.
cmd_vel은 어디서 만들어지나요?	수동 override, autonomy, coordinator 입력이 twist_mux를 거쳐 최종 /cmd_vel이 됩니다.
실차 3대가 되나요?	현재는 시뮬레이션과 일부 실차 연동 구조 검증 단계입니다. 완전한 3대 실차 자율주행은 확인 필요로 분리했습니다.
Pi 없이 ESP32 차량은 가능한가요?	관제 online, 수동 주행, E-Stop, 카메라 stream은 가능하지만 SLAM/Nav2는 중앙 PC compute와 센서 품질 검증이 필요합니다.

답변 원칙: 확인된 코드와 실행 결과를 먼저 말하고, 검증이 남은 부분은 '확인 필요' 또는 '향후 개선'으로 분리한다.

15. 확인됨 / 확인 필요 / 향후 개선

구분	항목
확인됨	AIP sim 웹관제 접속, 포트 18080/18765 분리, CONNECTED/MAP READY/3 online, 데모 주행, 참고형 map
확인됨	ROS2 패키지 구조, custom msg/srv, supervisor/watchdog, dashboard_server, sim_vehicle_node, serial_bridge 코드 경로
확인 필요	실차 3대 동시 완전 자율주행, 모든 차량의 heartbeat/odom/TF 안정성, E-Stop end-to-end 반복 검증
확인 필요	ESP32-S3 LiDAR LaserScan 품질, ESP32_CAM 지연, scout firmware FleetHeartbeat 필드 정합, motor PWM 매핑
향후 개선	WebSocket command ACK, 상태/오류 로그 UI, 실차 rosbag/MCAP 증거, 실차 mixed fleet 데모

포트폴리오에서 가장 중요한 것은 과장하지 않는 것이다. '한계까지 알고 있다'는 점이 신입 면접에서 오히려 강점이 된다.

16. 마지막 복습 루틴

면접 전 30분 루틴

- 웹관제 화면을 열고 CONNECTED, MAP READY, 3 online을 확인한다.
- AIP 시스템을 '공동 Topic 계약 - 중앙 집계 - 웹관제 - 차량 driver' 네 단계로 말해본다.
- 시뮬레이션과 실제 차량 launch가 분리되어 있음을 설명한다.
- ESP32 차량은 가능한 범위와 어려운 범위를 구분해 말한다.
- 실차 3대 완전 자율주행은 확인 필요라고 솔직히 말한다.

외워둘 문장

AIP 프로젝트는 ROS2 Topic 계약을 기준으로 이기종 로봇을 하나의 웹관제 화면에 묶는 구조를 실험한 프로젝트입니다. 현재는 시뮬레이션 데모와 일부 실차 확장 구조가 확인되었고, 완전한 3대 실차 자율주행은 향후 검증 과제로 분리했습니다.

근거 문서: README.md, PORTFOLIO_KO.md, docs/architecture/system-architecture.md, docs/architecture/ros2-communication.md, docs/architecture/web-dashboard.md, docs/architecture/sub-vehicle-control.md, docs/portfolio/interview-notes.md